

Enunciado Proyecto 1

Estructura de Datos y Algoritmos

1 Introducción

En este proyecto, cada estudiante deberá desarrollar una estrategia para resolver un juego de tablero original llamado **GridLink**, inspirado en rompecabezas clásicos de conexión de caminos como *Numberlink* o *Flow Free*, pero con reglas propias. El objetivo es construir conexiones entre pares de conectores revelados progresivamente durante la partida, administrando el espacio disponible del tablero y anticipando futuras solicitudes. Para ello, el jugador deberá decidir tanto la forma de cada conexión como el orden en que resuelve las solicitudes disponibles, enfrentándose a distintas modalidades de juego que privilegian la optimización del espacio o la capacidad de sobrevivir la mayor cantidad de rondas posible. La competencia se desarrollará en formato de torneo, y ganará quien obtenga el mayor puntaje acumulado al finalizar todas las partidas.

2 Descripción general de GridLink

GridLink se desarrolla sobre un tablero rectangular compuesto por distintos tipos de celdas: conectores, celdas normales, bloqueos, puentes y celdas potenciales. Desde el inicio de la partida, el jugador conoce la posición de todos los conectores, pero desconoce qué pares deberán unirse. Durante cada ronda se revelan una o más solicitudes de conexión y el jugador debe seleccionar una de ellas para construir un camino válido entre ambos conectores, respetando las restricciones del tablero. Dependiendo de la modalidad de juego, las conexiones previamente construidas podrán modificarse o permanecerán fijas durante toda la partida. Adicionalmente, algunas celdas potenciales pueden transformarse en puentes a medida que avanza el juego, permitiendo que dos conexiones compartan una misma posición del tablero.

La Figura 1 muestra un ejemplo de la evolución de una partida. Inicialmente se observan todos los conectores, bloqueos, puentes y celdas potenciales, mientras que solo una solicitud se encuentra disponible. En las rondas siguientes el jugador construye nuevas conexiones y el estado del tablero evoluciona conforme se agregan caminos, se incorporan nuevas solicitudes y aparecen nuevos puentes, reduciendo progresivamente el espacio disponible para las conexiones futuras.

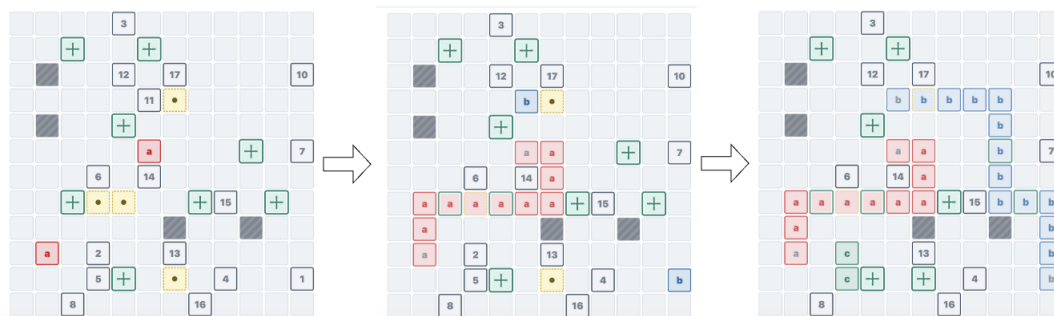


Figure 1: Ejemplo de dos jugadas, mostrando actualización del tablero.



2.1 Reglas

2.1.1 Estructura del tablero

- El tablero siempre es rectangular, con dimensiones definidas al inicio de cada partida. Estas dimensiones podrán variar entre 8×8 y 30×40 .
- Todas las celdas del tablero son visibles desde el inicio de la partida.
- Cada celda pertenece exactamente a uno de los siguientes tipos:
 - Celda normal.
 - Conector.
 - Bloqueo.
 - Puente.
 - Puente potencial.

2.1.2 Conectores

- Todos los conectores son visibles desde el inicio de la partida, pero inicialmente no se conoce qué pares deberán conectarse.
- Cada conector puede participar en una única conexión durante toda la partida.
- Un conector no puede ser atravesado por otra conexión.
- Todo conector debe tener al menos tres celdas vecinas ortogonales libres (sin bloqueos ni otros conectores).

2.1.3 Bloqueos y puentes

- Los bloqueos no pueden ser utilizados por ninguna conexión.
- La cantidad de bloqueos iniciales no puede superar el 10% de las celdas del tablero.
- Los puentes pueden ser utilizados simultáneamente por hasta dos conexiones distintas.
- Cada conexión debe entrar y salir del puente por lados distintos.
- La cantidad de puentes iniciales no puede superar el 20% de las celdas del tablero.
- Un puente inicial no puede ubicarse en la orilla del tablero ni tener bloqueos como vecinos ortogonales.
- Los puentes potenciales se comportan inicialmente como celdas normales.
- La cantidad de puentes potenciales no puede superar el 10% de las celdas del tablero.
- Los puentes potenciales tienen las mismas restricciones de ubicación que los puentes iniciales.



2.1.4 Construcción de conexiones

- Una conexión une exactamente dos conectores pertenecientes a una misma solicitud.
- Los caminos deben ser continuos.
- Sólo se permiten movimientos ortogonales (arriba, abajo, izquierda y derecha).
- No se permiten movimientos diagonales.
- Un camino no puede atravesar bloqueos.
- Un camino no puede atravesar otros conectores.
- Una celda normal sólo puede pertenecer a una conexión.
- Una celda puente puede pertenecer simultáneamente a dos conexiones.
- Los caminos no pueden ramificarse.

2.1.5 Solicitudes de conexión

- Cada partida define una secuencia fija de solicitudes.
- El número total de solicitudes nunca supera la mitad del número de conectores presentes en el tablero.
- Cada partida define una ventana de solicitudes de tamaño 1, 2 o 3.
- En cada ronda el jugador puede escoger cualquiera de las solicitudes visibles en la ventana.
- Al completar correctamente una solicitud, ésta se elimina de la ventana y se incorpora la siguiente solicitud pendiente de la secuencia.
- La cantidad de solicitudes disponibles será el máximo entre la ventana de la partida y la cantidad de solicitudes pendientes en el juego.

2.1.6 Transformación de puentes potenciales

- Cada partida define una probabilidad de transformación de puentes potenciales.
- Después de cada ronda completada se evalúa si ocurre una transformación.
- En cada ronda puede transformarse como máximo un puente potencial.
- Si ocurre el evento, se selecciona aleatoriamente una de las celdas potenciales restantes, la que pasa permanentemente a ser un puente.
- Si la celda potencial ya formaba parte de una conexión, dicha conexión permanece sin modificaciones y la celda pasa a tener capacidad para una segunda conexión.
- Si no quedan puentes potenciales, no pueden producirse nuevas transformaciones.



2.1.7 Modalidad Menos celdas

- El objetivo es completar todas las solicitudes minimizando el costo final de la solución.
- Antes de construir la nueva conexión, el jugador puede modificar libremente cualquiera de las conexiones existentes.
- Al finalizar cada ronda, todas las solicitudes previamente completadas deben continuar correctamente conectadas.
- El costo final se calcula como:

$$\text{costo} = \text{celdas finales} + N \times \text{celdas eliminadas}$$

donde $N \in \{0.5, 1, 1.5, 2\}$ corresponde a la penalización definida para la partida.

- Una celda puente utilizada por dos conexiones se contabiliza una única vez dentro de las celdas finales.
- Las celdas eliminadas se suman después de cada ronda, por ende, si una celda se elimina en una ronda, pero luego se utiliza en la siguiente, esa celda contó como eliminada.
- El puntaje obtenido en la partida se calcula como:

$$\text{puntaje} = \frac{\text{filas} \times \text{columnas} \times \text{solicitudes}^2}{\text{costo}}$$

donde *solicitudes* corresponde a la cantidad total de solicitudes de conexión definidas para la partida. De esta forma, las partidas con tableros más grandes o con un mayor número de solicitudes entregan puntajes potencialmente superiores, mientras que utilizar más celdas o realizar más modificaciones disminuye el puntaje obtenido.

2.1.8 Modalidad Supervivencia

- El objetivo es completar la mayor cantidad posible de solicitudes.
- Una vez construida una conexión, ésta no puede modificarse.
- La partida termina cuando ninguna de las solicitudes visibles puede resolverse mediante una conexión válida.
- El puntaje de la partida corresponde al cuadrado del número de rondas completadas.

2.1.9 Fin de la partida

- En la modalidad *Menos celdas*, la partida finaliza cuando se completan todas las solicitudes.
- En la modalidad *Supervivencia*, la partida finaliza cuando no es posible completar ninguna de las solicitudes visibles.
- En ambas modalidades, una jugada inválida o no entregar una jugada dentro del tiempo establecido implica la pérdida inmediata de la partida, obteniendo 0 puntos.



3 Aplicación del estudiante

Cada estudiante deberá desarrollar una aplicación *jugador automático* para *GridLink* que tome decisiones de jugada en base al estado del tablero y a las reglas del juego. La aplicación se ejecutará por lotes (múltiples partidas), y su objetivo será obtener la mayor cantidad de puntos por partida.

Interacción con la API (visión general). La aplicación se comunicará con el servidor mediante una API provista por el profesor. El servidor controlará la generación del tablero, la evolución del juego y el uso de semillas aleatorias para garantizar que todos los estudiantes jueguen exactamente las mismas partidas en el torneo. A un nivel conceptual, el ciclo de juego es:

1. **Obtener el estado de partida:** consultar el estado actual (estado del tablero, modalidad, parámetros de la partida).
2. **Decidir la jugada:** calcular la siguiente conexión, aplicando la estrategia del estudiante.
3. **Enviar la jugada:** remitir la acción al servidor y recibir la respuesta con el nuevo estado.
4. **Control de término:** repetir hasta que la partida finalice según la modalidad; registrar el puntaje.

Requisitos operativos. La aplicación debe:

- Soportar ejecución sobre múltiples partidas en modo torneo.
- Respetar un tiempo máximo de ejecución de **10 minutos para la resolución de un lote de 10 partidas** (tiempo medido de extremo a extremo).

4 Entregas

4.1 Entrega parcial [1.2 ptos.]

Miércoles 12 de agosto en repositorio. Durante la clase de ese día (13.30, presencial) se realizará un conjunto de juegos entre todos.

Para esta entrega su jugador deberá jugar juegos con:

- Modalidad Supervivencia.
- Ventanas siempre de 1.
- Sin puentes potenciales.

El puntaje se asignará de la siguiente forma:

- 1.2 $\rightarrow$ durante la clase se realizarán dos lotes de 8 juegos cada uno con diferentes parámetros. Para obtener todo el puntaje su jugador **debe** obtener un puntaje igual o superior a un valor N que será informado previamente al comienzo del torneo. Luego se descuenta una décima por cada 10 puntos menos o fracción.
- Bono: 0.3 $\rightarrow$ para el o los jugadores que tengan el mayor puntaje.



4.2 Entrega final [4.5 ptos.]

Miércoles 26 de agosto en repositorio (no tener el código actualizado en el repositorio implica un descuento de 1 punto en la nota de la entrega).

Durante la clase de ese mismo día (13.30, presencial) se realizará la competencia. El profesor desarrollará 2 jugadores, uno simple y uno avanzado, los cuáles también competirán. La nota será competitiva, obteniendo el máximo puntaje (4.5) aquel que obtenga el máximo puntaje total y supere además el puntaje del jugador avanzado del profesor. El jugador simple del profesor definirá el corte para obtener 2 puntos en la entrega (los que tengan menos puntos tendrán menos que eso). Por otro lado, para que el mejor jugador obtenga los 4.5 puntos deberá superar en puntaje al jugador avanzado del profesor.

La evaluación será sobre el resultado de los jugadores utilizando el juego con la API. No hay evaluación del código, ni de la intención o el esfuerzo. Es importante preocuparse entonces que su jugador funcione correctamente, maneje los casos de borde, y **esté preparado para una prueba intensa usando la red de la universidad.**

4.3 Entrega *post-mortem* [1.5 ptos.]

Miércoles 2 de septiembre por Canvas hasta las 10.00 horas. Esta consistirá en un documento con un análisis de su jugador y de su resultado en la evaluación. Las instrucciones detalladas se entregarán el martes 25/8 antes de la prueba de entrega final. De todas formas, les recomendamos generar logs de su participación en la entrega parcial y final, que les permitan luego hacer una evaluación de sus resultados.

4.4 Actividad Validación

El miércoles 2 de septiembre en clases (presencial) habrá una actividad individual donde deberán demostrar el conocimiento de su proyecto. Esta actividad no tiene puntaje, sino que fallar en ella puede llevar a un descuento de hasta 3 puntos en la nota final del proyecto.

5 Consideraciones

No hay restricciones respecto al desarrollo de su solución, pueden usar cualquier lenguaje, aplicación, programa, infraestructura, algoritmo, estrategia, ... Recuerde eso sí, que las pruebas serán en la universidad, por lo que es su responsabilidad contar con el equipamiento para poder realizarlas de **forma estrictamente presencial.**

El estudiante debe ser dueño de su código, debe comprender exactamente que está haciendo en cada momento.

No hay punto base, la nota se calcula como la suma del puntaje obtenido en las tres instancias.

El problema es simple, lo pueden realizar en unas cuantas horas, pero la base está en la competitividad por ser el más eficiente (máximo puntaje).